9.4 Publish IMU and Odometer Data

In robot navigation, accurately calculating real-time position is essential. Normally, we obtain odometer information using motor encoders and the robot's kinematic model. However, in specific situations, like when the robot's wheels rotate in place or when the robot is lifted, it may move a distance without the wheels actually turning.

To address wheel slip or accumulated errors in such cases, combining IMU and odometer data can yield more precise odometer information. This improves mapping and navigation accuracy in scenarios where wheel slip or cumulative errors may occur.

1. Introduction to IMU and Odometer

The IMU (Inertial Measurement Unit) is a device that measures the three-axis attitude angles (angular velocity) and acceleration of an object. It consists of the gyroscope and accelerometer as its main components, providing a total of 6 degrees of freedom to measure the object's angular velocity and acceleration in three-dimensional space.

An odometer is a method used to estimate changes in an object's position over time using data obtained from motion sensors. This method is widely applied in robotic systems to estimate the distance traveled by the robot relative to its initial position.

There are common methods for odometer positioning, including the wheel odometer, visual odometer, and visual-inertial odometer. In robotics, we specifically use the wheel odometer. To illustrate the principle of the wheel odometer, consider a carriage where you want to determine the distance from point A to point B. By knowing the circumference of the carriage wheels and installing a device to count wheel revolutions, you can calculate the distance

based on wheel circumference, time taken, and the number of wheel revolutions.

While the wheel odometer provides basic pose estimation for wheeled robots, it has a significant drawback: accumulated error. In addition to inherent hardware errors, environmental factors such as slippery tires due to weather conditions contribute to increasing odometer errors with the robot's movement distance.

Therefore, both IMU and odometer are essential components in a robot. These two components are utilized to measure the three-axis attitude angles (or angular velocity) and acceleration of the object, as well as to estimate the distance, pose, velocity, and direction of the robot relative to its initial position. To address these errors, we combine IMU data with odometer data to obtain more accurate information. IMU data is published through the "/imu" topic, and odometer data is published through "/odom". After obtaining data from both sources, the data is fused using the "ekf" package in ROS, and the fused localization information is then republished.

1.1 IMU Data Publishing

1.1.1 Initiate Service

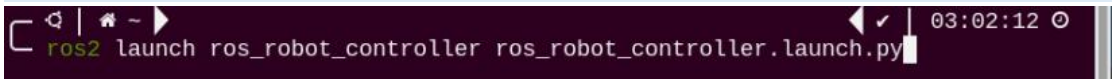
Note: The input command is case-sensitive, and keywords can be completed using the Tab key.

1) Start the robot, and access the robot system desktop using VNC.
2) Click-on  to open the command-line terminal.
3) Execute the command, and hit Enter to disable the app auto-start service.

```
~/.stop_ros.sh
```

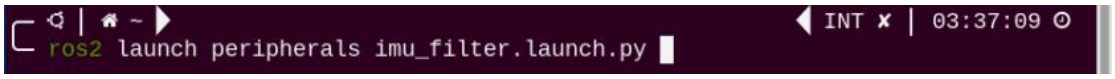
4) Enter the command and press Enter to enable the chassis control node.

```
ros2 launch ros_robot_controller ros_robot_controller.launch.py
```



5) Create a new command line terminal. Enter the command and press Enter to publish the IMU data.

```
ros2 launch peripherals imu_filter.launch.py
```



1.1.2 Data Viewing

1) Open a new command line terminal, and execute the command to check the current topic.

```
ros2 topic list
```

2) Enter the command to view the type, publisher, and subscribers of the "**/imu**" topic. You can replace "**/imu**" with the topic you want to view. The type of this topic is "**sensor_msgs/msg/Imu**".

```
ros2 topic info /imu
```

3) Use the following command to display the content of the topic message.

Feel free to replace '**imu**' with the name of the topic you wish to view.

```
ros2 topic echo /imu
```

```
> ros2 topic echo /imu
```

```
angular_velocity:
  x: -0.0007023881084129262
  y: -0.0012877124562211285
  z: -0.0006172497924044479
angular_velocity_covariance:
- 0.01
- 0.0
- 0.0
- 0.0
- 0.01
- 0.0
- 0.0
- 0.0
- 0.01
linear_acceleration:
  x: 1.2622444743093562
  y: -0.24267646318977404
  z: -12.979330169014995
linear_acceleration_covariance:
- 0.0004
- 0.0
```

The terminal will display the data from the three axes of the IMU.

4) If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

5) Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```

1.2 Odometer Data Publishing

1.2.1 Initiate Service

Note: The input command is case-sensitive, and keywords can be completed using the Tab key.

1) Start the robot, and connect it to the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Enter the command and hit Enter to disable the app auto-start service.

~/.stop_ros.sh

1) Run the command to publish the odometer data.

ros2 launch controller odom_publisher.launch.py

1.2.2 Data Viewing

1) Open a new command line terminal, and run the command below to check the current topic.

ros2 topic list

2) Enter the command to view the type, publisher, and subscribers of the "**/odom_raw**" topic. You can replace "**/odom_raw**" with the topic you want to view. The type of this topic is "**nav_msgs/msg/Odometry**".

ros2 topic info /odom_raw

```
> ros2 topic info /odom_raw
Type: nav_msgs/msg/Odometry
Publisher count: 1
Subscription count: 0
```

3) Enter the command to print the topic message contents. You can replace the topic you want to view as needed.

ros2 topic echo /odom_raw

```
ros2 topic echo /odom_raw
```

```
> ros2 topic echo /odom_raw
header:
  stamp:
    sec: 3722
    nanosec: 936043488
  frame_id: odom
child_frame_id: base_footprint
pose:
  pose:
    position:
      x: 0.0
      y: 0.0
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.0
      w: 1.0
  covariance:
  - 1.0e-09
  - 0.0
  - 0.0
  - 0.0
  - 0.0
```

The message content includes acquired pose and velocity data.

4) If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

5) Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

sudo systemctl restart start_node.service